Energy Consumption Drivers Exploratory Data Analysis

Ohenmaa Sarfo-Bonsu

2025-04-21

Introduction

```r
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3

## Warning: package 'tibble' was built under R version 4.4.3

## Warning: package 'tidyr' was built under R version 4.4.3

## Warning: package 'readr' was built under R version 4.4.3

## Warning: package 'purrr' was built under R version 4.4.3

## Warning: package 'dplyr' was built under R version 4.4.3

## Warning: package 'lubridate' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(ggplot2)
library(dplyr)
library(rlang)
```

```
## Warning: package 'rlang' was built under R version 4.4.3
```

```
##
## Attaching package: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##     flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##     flatten_raw, invoke, splice
```

Below I have loaded the data tables and named them accordingly.

```r
train <- read.csv("train.csv")
gas_prices <- read.csv("gas_prices.csv")
client<- read.csv("client.csv")
electricity_prices <- read.csv("electricity_prices.csv")
forecast_weather<- read.csv("forecast_weather.csv")
historical_weather<-read.csv("historical_weather.csv")
```

Data Cleaning

```r
sum(is.na(train))
```

```
## [1] 528
```

```r
sum(is.na(gas_prices))
```

```
## [1] 0
```

```r
sum(is.na(client))
```

```
## [1] 0
```

```r
sum(is.na(electricity_prices))
```

```
## [1] 0
```

```r
sum(is.na(forecast_weather))
```

```
## [1] 2
```

```r
sum(is.na(historical_weather))
```

```
## [1] 0
```

```r
colSums(is.na(train))
```

```
##             county        is_business       product_type             target
##                  0                  0                  0                528
##     is_consumption           datetime      data_block_id             row_id
##                  0                  0                  0                  0
## prediction_unit_id
##                  0
```

```r
colSums(is.na(forecast_weather))
```

```
##                          latitude                         longitude
##                                 0                                 0
##                   origin_datetime                       hours_ahead
##                                 0                                 0
##                       temperature                          dewpoint
##                                 0                                 0
##                   cloudcover_high                    cloudcover_low
##                                 0                                 0
##                    cloudcover_mid                  cloudcover_total
##                                 0                                 0
##        X10_metre_u_wind_component        X10_metre_v_wind_component
##                                 0                                 0
##                     data_block_id                 forecast_datetime
##                                 0                                 0
##            direct_solar_radiation surface_solar_radiation_downwards
##                                 0                                 2
##                          snowfall                total_precipitation
##                                 0                                 0
```

```r
# Remove NA rows
train <- na.omit(train)
forecast_weather <- na.omit(forecast_weather)
```

```r
summary(train)
```

```
##      county        is_business      product_type       target
##  Min.   : 0.000   Min.   :0.0000   Min.   :0.000   Min.   :    0.000
##  1st Qu.: 3.000   1st Qu.:0.0000   1st Qu.:1.000   1st Qu.:    0.378
##  Median : 7.000   Median :1.0000   Median :2.000   Median :   31.133
##  Mean   : 7.297   Mean   :0.5368   Mean   :1.899   Mean   :  274.856
##  3rd Qu.:11.000   3rd Qu.:1.0000   3rd Qu.:3.000   3rd Qu.:  180.206
##  Max.   :15.000   Max.   :1.0000   Max.   :3.000   Max.   :15480.274
##  is_consumption   datetime         data_block_id       row_id
##  Min.   :0.0    Length:2017824     Min.   :  0.0   Min.   :      0
##  1st Qu.:0.0    Class :character   1st Qu.:166.0   1st Qu.: 504582
##  Median :0.5    Mode  :character   Median :323.0   Median :1009172
##  Mean   :0.5                       Mean   :321.9   Mean   :1009178
##  3rd Qu.:1.0                       3rd Qu.:479.0   3rd Qu.:1513763
##  Max.   :1.0                       Max.   :637.0   Max.   :2018351
##  prediction_unit_id
##  Min.   : 0.00
##  1st Qu.:16.00
##  Median :33.00
```

```
##  Mean   :33.05
##  3rd Qu.:50.00
##  Max.   :68.00
```

```r
summary(gas_prices)
```

```
##  forecast_date      lowest_price_per_mwh highest_price_per_mwh
##  Length:637         Min.   : 28.10       Min.   : 34.00
##  Class :character   1st Qu.: 60.00       1st Qu.: 67.53
##  Mode  :character   Median : 85.21       Median : 93.47
##                     Mean   : 95.04       Mean   :107.75
##                     3rd Qu.:109.00       3rd Qu.:130.74
##                     Max.   :250.00       Max.   :305.00
##  origin_date        data_block_id
##  Length:637         Min.   :  1
##  Class :character   1st Qu.:160
##  Mode  :character   Median :319
##                     Mean   :319
##                     3rd Qu.:478
##                     Max.   :637
```

```r
summary(client)
```

```
##   product_type       county         eic_count       installed_capacity
##  Min.   :0.000   Min.   : 0.000   Min.   :   5.00   Min.   :    5.5
##  1st Qu.:1.000   1st Qu.: 3.000   1st Qu.:  13.00   1st Qu.:  321.9
##  Median :2.000   Median : 7.000   Median :  32.00   Median :  645.2
##  Mean   :1.899   Mean   : 7.297   Mean   :  73.35   Mean   : 1450.8
##  3rd Qu.:3.000   3rd Qu.:11.000   3rd Qu.:  70.00   3rd Qu.: 1567.2
##  Max.   :3.000   Max.   :15.000   Max.   :1517.00   Max.   :19314.3
##   is_business         date           data_block_id
##  Min.   :0.0000   Length:41919       Min.   :  2.0
##  1st Qu.:0.0000   Class :character   1st Qu.:167.0
##  Median :1.0000   Mode  :character   Median :324.0
##  Mean   :0.5368                      Mean   :322.9
##  3rd Qu.:1.0000                      3rd Qu.:480.0
##  Max.   :1.0000                      Max.   :637.0
```

```r
summary(electricity_prices)
```

```
##  forecast_date      euros_per_mwh     origin_date        data_block_id
##  Length:15286       Min.   : -10.06   Length:15286       Min.   :  1
##  Class :character   1st Qu.:  85.29   Class :character   1st Qu.:160
##  Mode  :character   Median : 128.28   Mode  :character   Median :319
##                     Mean   : 157.06                      Mean   :319
##                     3rd Qu.: 199.80                      3rd Qu.:478
##                     Max.   :4000.00                      Max.   :637
```

```r
summary(forecast_weather)
```

```
##     latitude       longitude     origin_datetime     hours_ahead
```

```
##  Min.   :57.60   Min.   :21.70   Length:3424510     Min.   : 1.00
##  1st Qu.:57.98   1st Qu.:23.20   Class :character   1st Qu.:13.00
##  Median :58.50   Median :25.20   Mode  :character   Median :25.00
##  Mean   :58.65   Mean   :24.95                      Mean   :24.50
##  3rd Qu.:59.10   3rd Qu.:26.70                      3rd Qu.:36.75
##  Max.   :59.70   Max.   :28.20                      Max.   :48.00
##   temperature          dewpoint       cloudcover_high   cloudcover_low
##  Min.   :-27.4994   Min.   :-29.684   Min.   :0.00000   Min.   :0.0000000
##  1st Qu.:  0.2606   1st Qu.: -2.364   1st Qu.:0.00000   1st Qu.:0.0003357
##  Median :  4.8727   Median :  1.836   Median :0.08867   Median :0.2305491
##  Mean   :  5.7439   Mean   :  2.412   Mean   :0.39467   Mean   :0.4346456
##  3rd Qu.: 11.1464   3rd Qu.:  7.302   3rd Qu.:0.97849   3rd Qu.:0.9994587
##  Max.   : 31.8107   Max.   : 23.681   Max.   :1.00001   Max.   :1.0000076
##  cloudcover_mid   cloudcover_total X10_metre_u_wind_component
##  Min.   :0.0000   Min.   :0.0000   Min.   :-17.577
##  1st Qu.:0.0000   1st Qu.:0.2649   1st Qu.: -1.467
##  Median :0.1012   Median :0.9773   Median :  1.469
##  Mean   :0.3591   Mean   :0.6820   Mean   :  1.255
##  3rd Qu.:0.9001   3rd Qu.:1.0000   3rd Qu.:  3.814
##  Max.   :1.0000   Max.   :1.0000   Max.   : 22.573
##  X10_metre_v_wind_component data_block_id forecast_datetime
##  Min.   :-22.1161           Min.   :  1   Length:3424510
##  1st Qu.: -1.9781           1st Qu.:160   Class :character
##  Median :  0.9433           Median :319   Mode  :character
##  Mean   :  0.7250           Mean   :319
##  3rd Qu.:  3.5076           3rd Qu.:478
##  Max.   : 19.3144           Max.   :637
##  direct_solar_radiation surface_solar_radiation_downwards    snowfall
##  Min.   : -0.7733       Min.   : -0.3258                  Min.   :-3.815e-06
##  1st Qu.:  0.0000       1st Qu.:  0.0000                  1st Qu.: 0.000e+00
##  Median :  0.0000       Median :  0.6044                  Median : 0.000e+00
##  Mean   :151.1882       Mean   :110.7642                  Mean   : 2.534e-05
##  3rd Qu.:212.8451       3rd Qu.:144.1723                  3rd Qu.: 0.000e+00
##  Max.   :954.4222       Max.   :848.7144                  Max.   : 4.833e-03
##  total_precipitation
##  Min.   :-1.529e-05
##  1st Qu.: 0.000e+00
##  Median : 0.000e+00
##  Mean   : 7.864e-05
##  3rd Qu.: 2.766e-05
##  Max.   : 1.652e-02
```

```r
summary(historical_weather)
```

```
##    datetime          temperature         dewpoint           rain
##  Length:1710802     Min.   :-23.700   Min.   :-25.90   Min.   : 0.00000
##  Class :character   1st Qu.:  0.000   1st Qu.: -2.60   1st Qu.: 0.00000
##  Mode  :character   Median :  5.100   Median :  1.70   Median : 0.00000
##                     Mean   :  5.741   Mean   :  2.24   Mean   : 0.04962
##                     3rd Qu.: 11.200   3rd Qu.:  7.20   3rd Qu.: 0.00000
##                     Max.   : 32.600   Max.   : 23.80   Max.   :16.80000
##     snowfall       surface_pressure cloudcover_total cloudcover_low
##  Min.   :0.00000   Min.   : 942.9   Min.   :  0.00   Min.   :  0.00
##  1st Qu.:0.00000   1st Qu.:1001.5   1st Qu.: 25.00   1st Qu.:  3.00
```

```
##  Median :0.00000   Median :1010.4   Median : 72.00   Median : 39.00
##  Mean   :0.01605   Mean   :1009.3   Mean   : 60.91   Mean   : 46.69
##  3rd Qu.:0.00000   3rd Qu.:1018.0   3rd Qu.:100.00   3rd Qu.: 94.00
##  Max.   :2.66000   Max.   :1049.3   Max.   :100.00   Max.   :100.00
##  cloudcover_mid   cloudcover_high  windspeed_10m    winddirection_10m
##  Min.   :  0.00   Min.   :  0.00   Min.   : 0.000   Min.   :  0.0
##  1st Qu.:  0.00   1st Qu.:  0.00   1st Qu.: 3.000   1st Qu.:139.0
##  Median : 16.00   Median : 10.00   Median : 4.500   Median :208.0
##  Mean   : 34.41   Mean   : 36.05   Mean   : 4.850   Mean   :197.9
##  3rd Qu.: 72.00   3rd Qu.: 85.00   3rd Qu.: 6.278   3rd Qu.:263.0
##  Max.   :100.00   Max.   :100.00   Max.   :21.750   Max.   :360.0
##  shortwave_radiation direct_solar_radiation diffuse_radiation    latitude
##  Min.   :  0.0       Min.   :  0.00         Min.   :  0.00    Min.   :57.60
##  1st Qu.:  0.0       1st Qu.:  0.00         1st Qu.:  0.00    1st Qu.:57.90
##  Median :  1.0       Median :  0.00         Median :  1.00    Median :58.50
##  Mean   :106.5       Mean   : 64.45         Mean   : 42.04    Mean   :58.65
##  3rd Qu.:140.0       3rd Qu.: 47.00         3rd Qu.: 74.00    3rd Qu.:59.10
##  Max.   :849.0       Max.   :754.00         Max.   :388.00    Max.   :59.70
##    longitude     data_block_id
##  Min.   :21.70   Min.   :  1.0
##  1st Qu.:23.20   1st Qu.:160.0
##  Median :24.70   Median :319.0
##  Mean   :24.95   Mean   :319.3
##  3rd Qu.:26.70   3rd Qu.:478.0
##  Max.   :28.20   Max.   :637.0
```

```
# No unusual values spotted
```

Analysis 1: Forecasted Weather vs Historical Weather

Module(s) highlighted: Module 3

What is the relationship between the forecasted weather and the historical weather?

The forecasted weather is the weather predicted and the historical weather is the actual observed weather.

Given that weather predictions are available for many hours ahead of the forecasted time, identify the number of hours in advance at which there is a significant drop in accuracy.

I started by using the mutating join function, left_join, to combine the forecast_weather data frame with the historical_weather data frame. The question asks for insight on the accuracy of the weather, so with the combined data frame, I grouped the rows by hour and created new columns reporting the mean of the particular differences in weather.

Given the columns that forecast_weather and historical_weather share, I decided to analyze the accuracy of the weather in terms of temperature and dewpoint, since the variables exhibit values that are close enough to be logically overlayed on the same graph.

```
# Group the data frame by hours_ahead.
# Create a new column reporting the difference of the particular weather trait.
# Summarize the results from each group, reporting the mean of the differences.
```

```r
# The lower the difference, the higher the accuracy.

summary <- forecast_weather |>
  select(latitude, longitude, hours_ahead, temperature, dewpoint, forecast_datetime) |>
  left_join(historical_weather,
            by = c( "latitude" = "latitude",
                    "longitude" = "longitude",
                    "forecast_datetime" = "datetime"
                    )) |>
  select(latitude, longitude, hours_ahead, temperature.x, temperature.y, dewpoint.x, dewpoint.y, foreca
  group_by(hours_ahead) |>
  mutate(temp_diff = abs(temperature.x - temperature.y),
         dewp_diff = abs(dewpoint.x - dewpoint.y)) |>
  summarize(mean_temp_diff = mean(temp_diff, na.rm = TRUE),
            mean_dewp_diff = mean(dewp_diff, na.rm = TRUE),
            n = n()
            )
```

```
## Warning in left_join(select(forecast_weather, latitude, longitude, hours_ahead, : Detected an unexpe
## i Row 2348082 of ‘x‘ matches multiple rows in ‘y‘.
## i Row 3025 of ‘y‘ matches multiple rows in ‘x‘.
## i If a many-to-many relationship is expected, set ‘relationship =
##   "many-to-many"‘ to silence this warning.
```

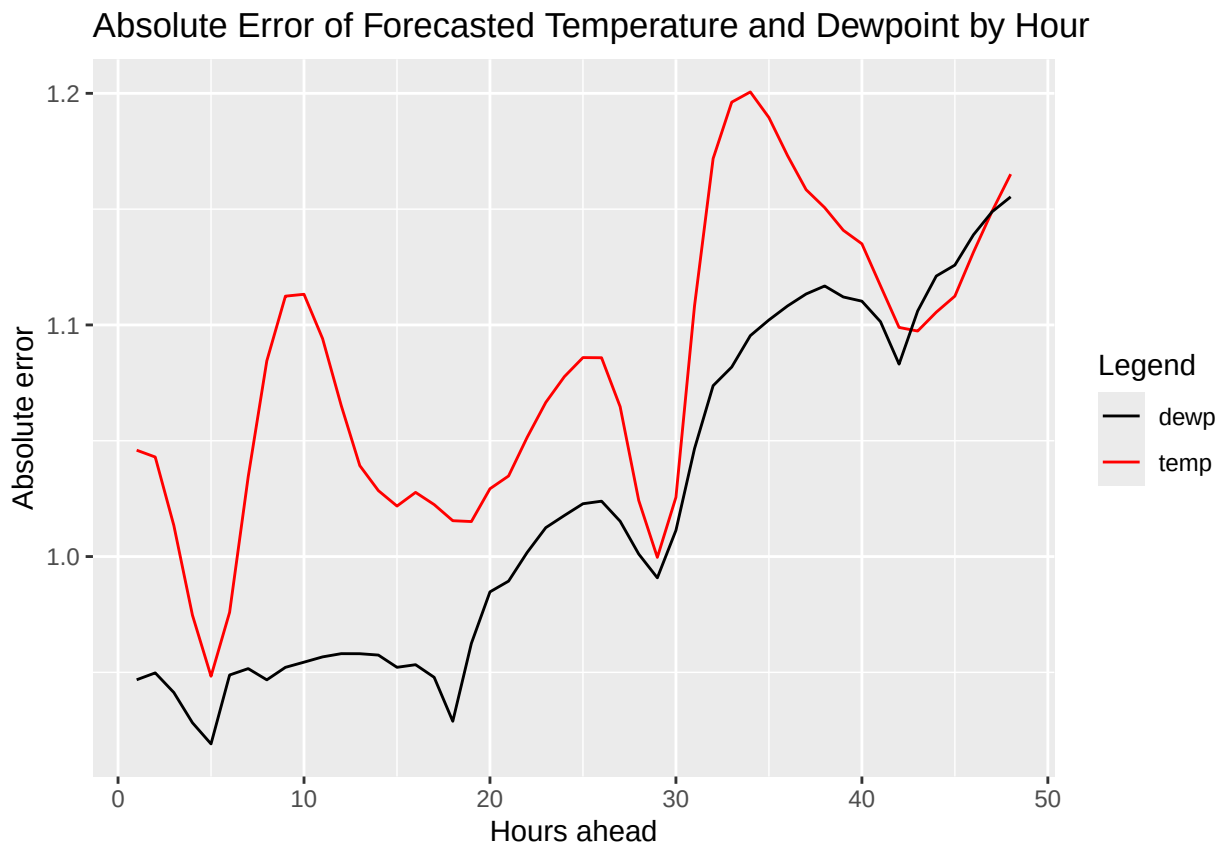
```r
summary
```

```
## # A tibble: 48 x 4
##    hours_ahead mean_temp_diff mean_dewp_diff     n
##          <int>          <dbl>          <dbl> <int>
##  1           1          1.05           0.947 71344
##  2           2          1.04           0.950 71344
##  3           3          1.01           0.941 71343
##  4           4          0.975          0.928 71343
##  5           5          0.948          0.919 71344
##  6           6          0.976          0.949 71344
##  7           7          1.03           0.952 71344
##  8           8          1.08           0.947 71344
##  9           9          1.11           0.952 71344
## 10          10          1.11           0.954 71344
## # i 38 more rows
```

I then used the data frame to create a graph in order to observe how the accuracy changed by the hour ahead.

```r
colors <- c("temp" = "red", "dewp" = "black")

summary |>
  ggplot() +
  geom_line(aes(x = hours_ahead, y = mean_temp_diff, color = "temp")) +
  geom_line(aes(x = hours_ahead, y = mean_dewp_diff, color = "dewp")) +
```

```r
labs(title = "Absolute Error of Forecasted Temperature and Dewpoint by Hour",
     x = "Hours ahead",
     y = "Absolute error",
     color = "Legend") +
scale_color_manual(values = colors)
```



The graph above reports the absolute error of the forecasted temperature and the forecasted dewpoint for up to 48 hours ahead. For this reason, the lower the absolute error, the higher the accuracy. The hours where the accuracy significantly drops are the places where the curve peaks. The hours in which the temperature peaks are 10, 25, and about 33. The hours in which the dewpoint peaks are about 12, 25, and 37. Therefore, the hours where the accuracy of the forecasted temperature and dewpoint drops are relatively close in value. There appears to be a trend of low accuracy revolving around those particular time stamps.

Analysis 2: Client Dataframe Analysis

Module(s) highlighted: Module 1, 2, 4

Is there a relationship between installed_capacity, the product type, and whether or not a client is a business? Does the relationship change over time?

My first inclination was to check if the relationship between the variables had to do with to the quantity of the product types per business. A quick data frame construction invalidated that idea.

```r
client |>
  select(product_type, installed_capacity, is_business, date) |>
  group_by(is_business) |>
  distinct(product_type, n())
```

```
## # A tibble: 7 x 3
## # Groups:   is_business [2]
##   is_business product_type `n()`
##         <int>        <int> <int>
## 1           0            1 41919
## 2           0            2 41919
## 3           0            3 41919
## 4           1            0 41919
## 5           1            1 41919
## 6           1            3 41919
## 7           1            2 41919
```

I then decided to investigate the measures of central tendencies for installed capacity per product type and business boolean. To achieve this efficiently, I created a function to compute the mean, median, and range of the installed capacity within the groups I formerly specified.

```r
# changing the business boolean to correspond to either "TRUE" or "FALSE"
client <- client |>
  mutate(is_business = case_when(
    is_business == 1 ~ "TRUE",
    is_business == 0 ~ "FALSE")

  )

# function to group by business and product type and return a particular measure of central tendency of
IC_summary <- function(measure) {
  client |>
  select(product_type, installed_capacity, is_business, date) |>
  group_by(is_business, product_type) |>
  summarize({{measure}}:= measure(installed_capacity), .groups = "drop")
}

# my.range function: computing the difference between max and min element in a vector
my.range <- function(vector) {
  max(vector) - min(vector)
}

# apply function to find mean, median, and my.range
mean_IC_summary<- IC_summary(mean)
median_IC_summary <- IC_summary(median)
my.range_IC_summary<- IC_summary(my.range)

mean_IC_summary
```

```
## # A tibble: 7 x 3
##   is_business product_type   mean
##   <chr>              <int>  <dbl>
```

```
## 1 FALSE                  1  603.
## 2 FALSE                  2   77.7
## 3 FALSE                  3 2149.
## 4 TRUE                   0  920.
## 5 TRUE                   1  664.
## 6 TRUE                   2  100.
## 7 TRUE                   3 2743.
```

```
median_IC_summary
```

```
## # A tibble: 7 x 3
##   is_business product_type median
##   <chr>              <int>  <dbl>
## 1 FALSE                  1   381
## 2 FALSE                  2    60
## 3 FALSE                  3  1241.
## 4 TRUE                   0   720
## 5 TRUE                   1   388.
## 6 TRUE                   2    92
## 7 TRUE                   3  1477.
```

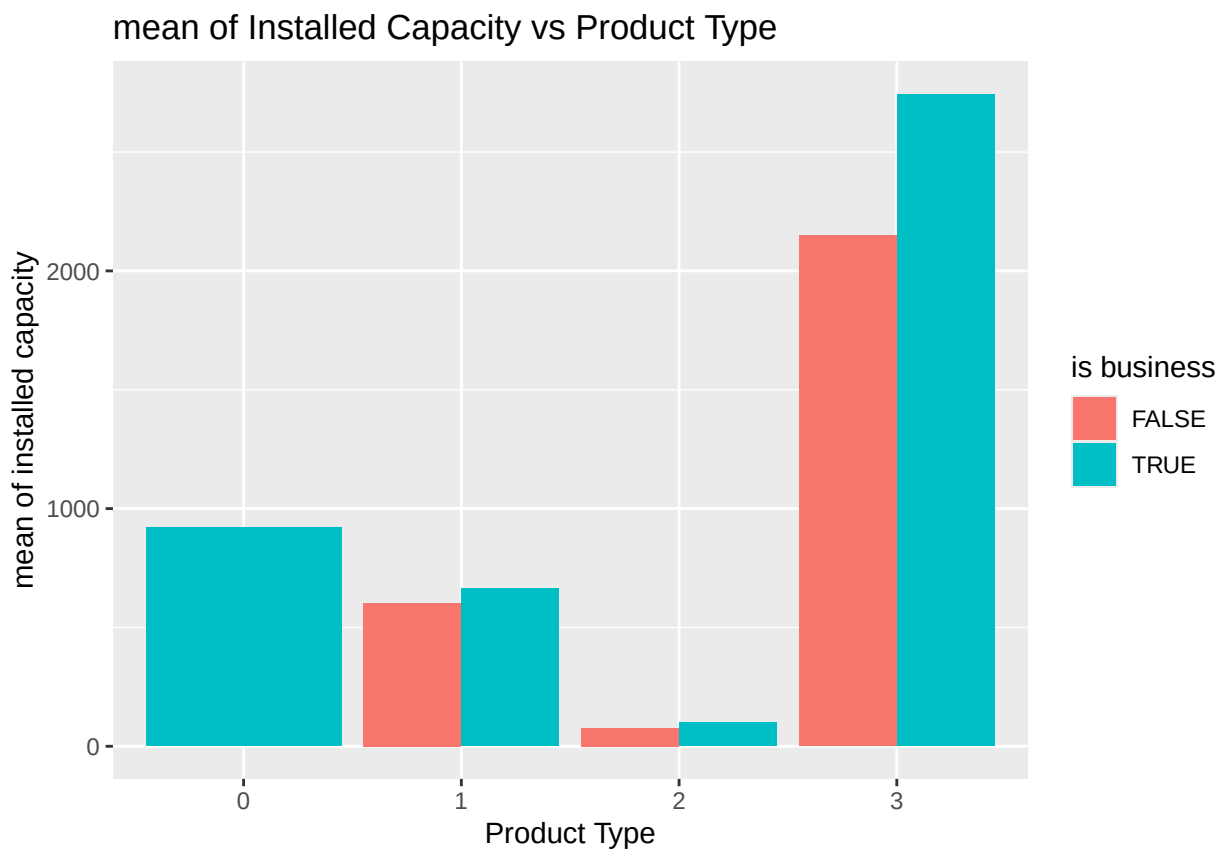
```
my.range_IC_summary
```

```
## # A tibble: 7 x 3
##   is_business product_type my.range
##   <chr>              <int>    <dbl>
## 1 FALSE                  1    5183.
## 2 FALSE                  2     210.
## 3 FALSE                  3   15601.
## 4 TRUE                   0    4709.
## 5 TRUE                   1    2841.
## 6 TRUE                   2     298.
## 7 TRUE                   3   19149.
```

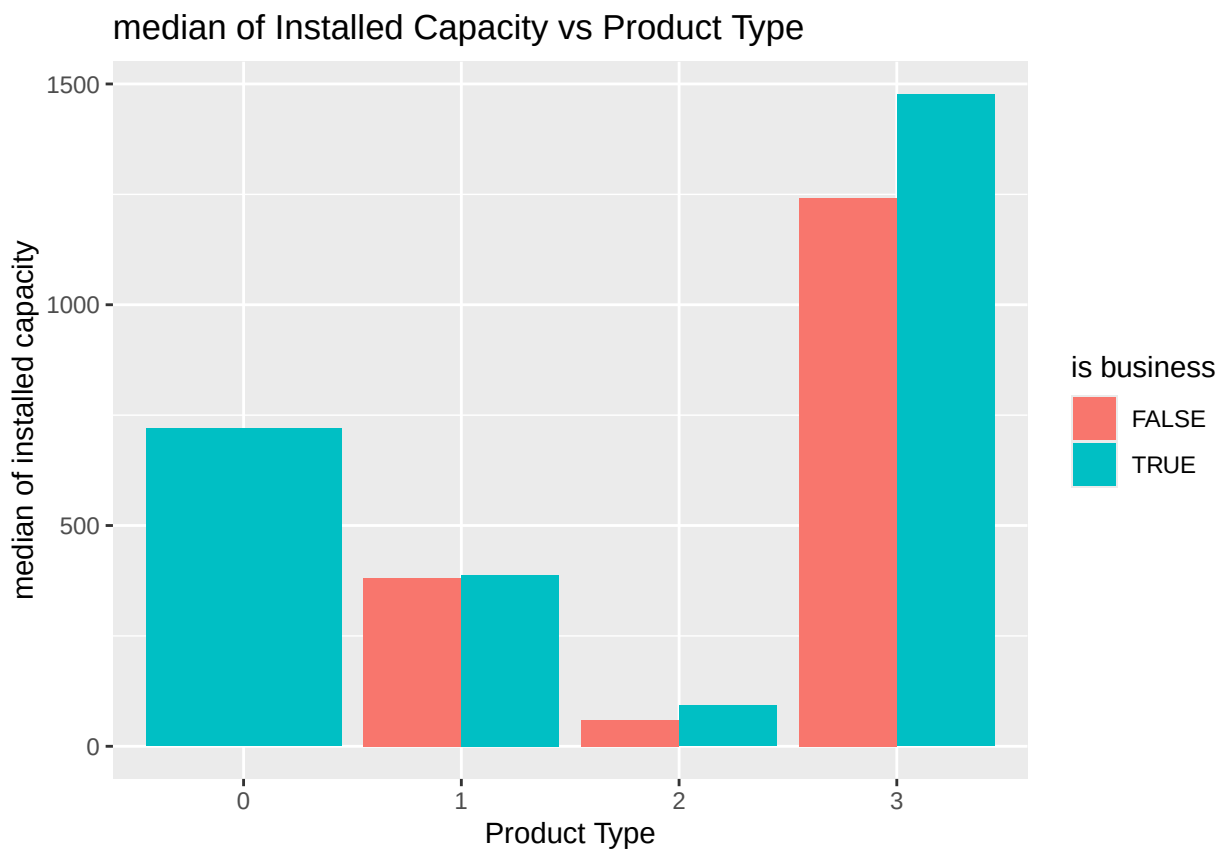
I then used a function to efficiently graph the mean, median, and range of the installed capacities and looked for patterns within the graphs.

```
graph_IC_summary <- function(df, measure) {
  title <- rlang::englue("{{measure}} of Installed Capacity vs Product Type")
  y <- rlang::englue("{{measure}} of installed capacity")
  df|>
  ggplot(aes(x = as.factor(product_type), y = {{measure}})) +
  geom_col(aes(fill = is_business), position = "dodge") +


  labs(title = title,
       x = "Product Type",
       y = y,
       fill = "is business"
       )
}
```

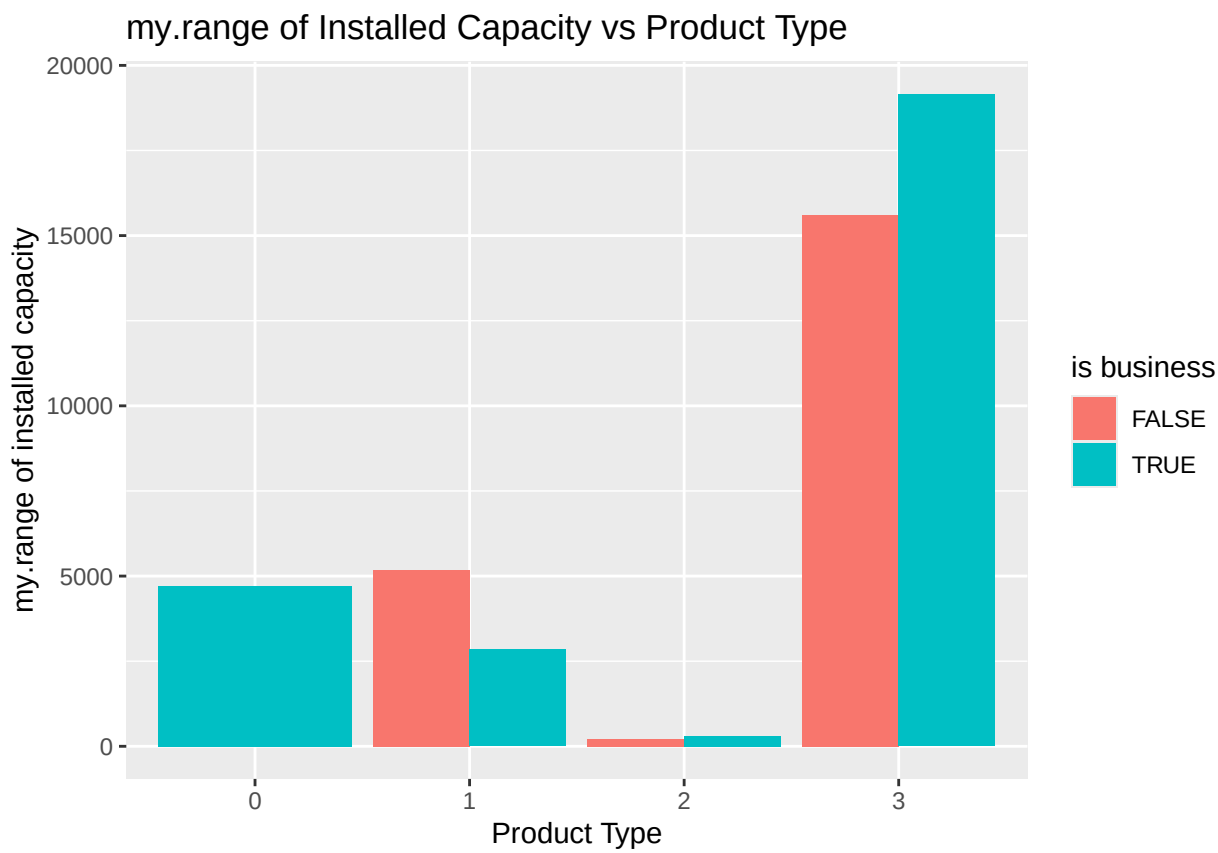
```r
# apply function to find graphs comparing mean, median, and range

graph_IC_summary(mean_IC_summary, mean)
```




```r
graph_IC_summary(median_IC_summary, median)
```




```
graph_IC_summary(my.range_IC_summary, my.range)
```

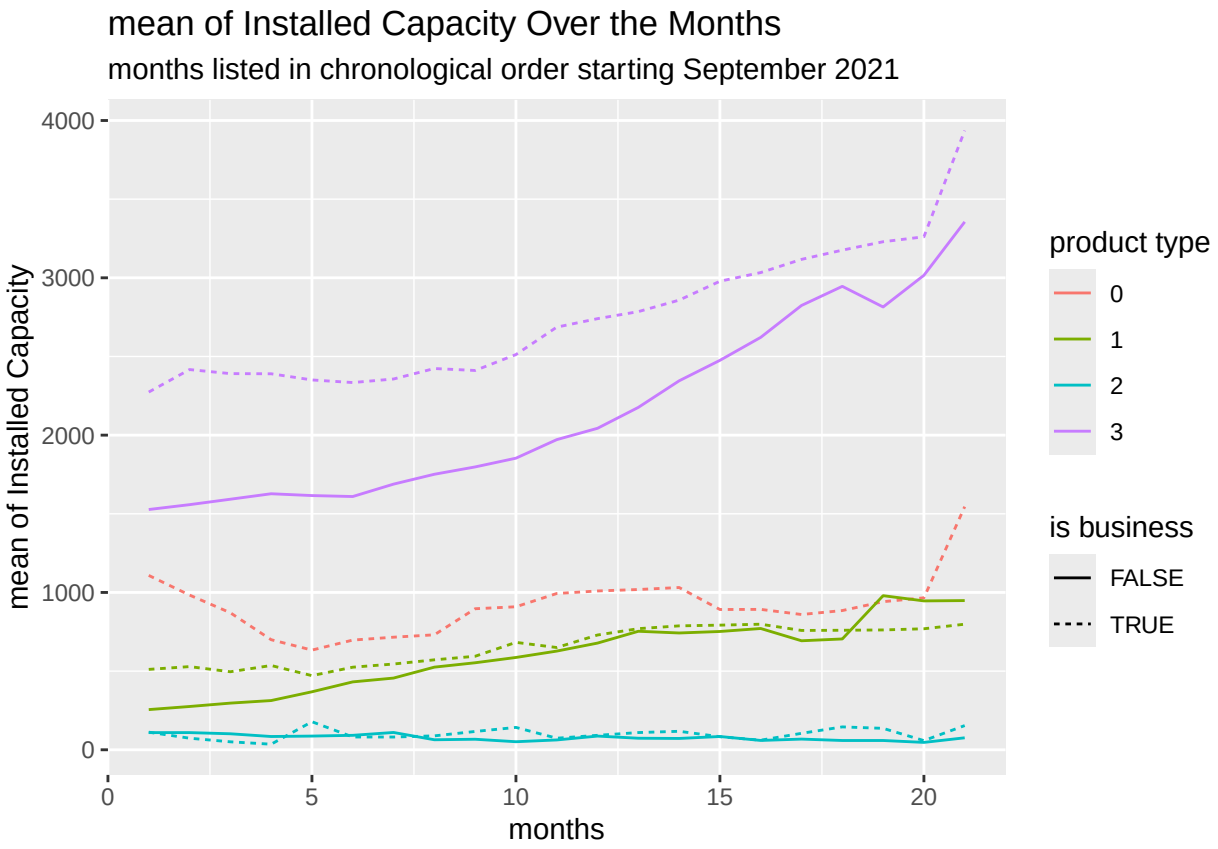


It appears that overall, the mean and median installed capacity for business clients are greater per product type compared to the non-business clients. The range of the installed capacity for business clients is greater than non-business clients for product types two and three, while the opposite is true for product type one.

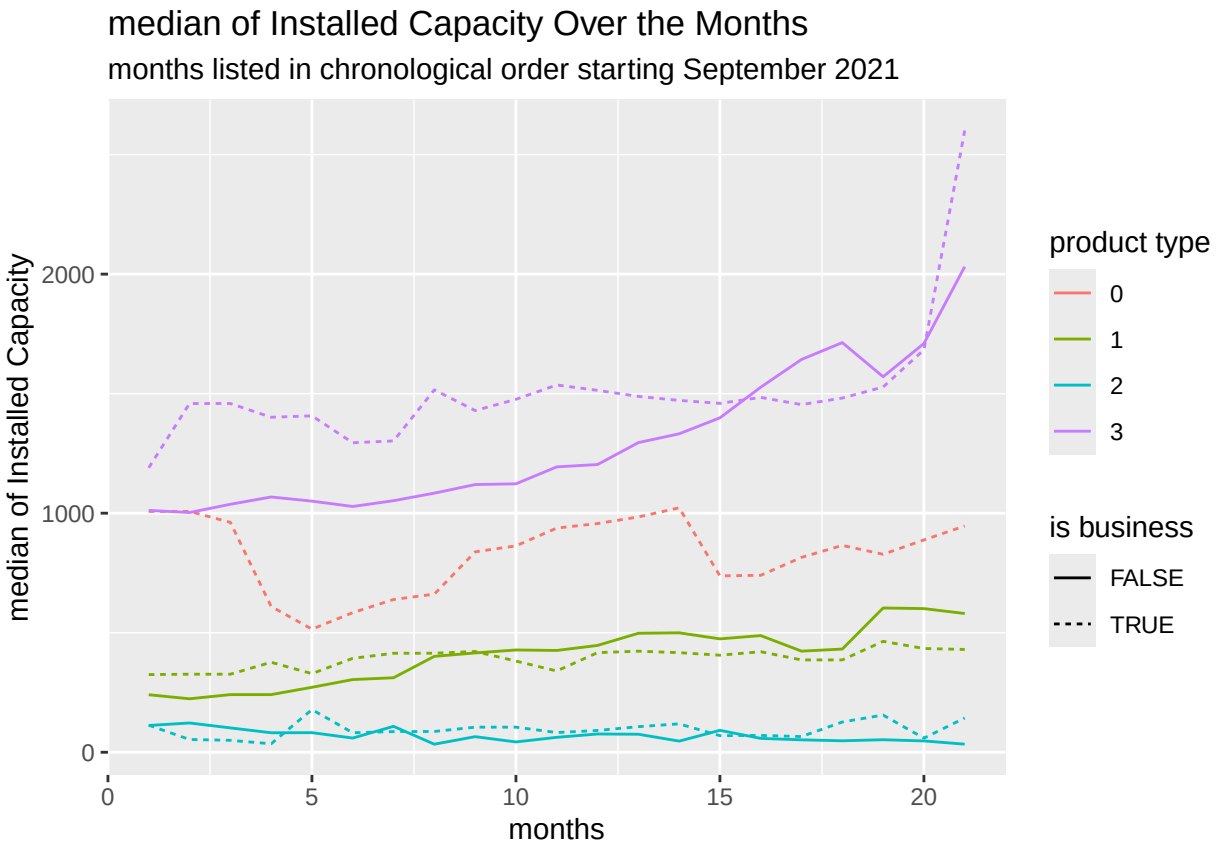
I then analyzed the relationship further. Namely, I used functions to create and graph data frames that investigated whether or not the mean, median, and range exhibited any notable changes overtime.

```
# assigning rank to order the year/month combinations to order them chronologically
rank<- client |>
  select(product_type, installed_capacity, is_business, date) |>
  mutate(date = ymd(date),
         year = year(date),
         month = month(date)
         ) |>
  group_by(year, month) |>
  summarize(mean_IC = mean(installed_capacity), .groups = "drop") |>
  select(!mean_IC)|>
  mutate(chron_month = row_number())


# function to create data frame tracking how mean/median/range changes overtime

overtime_df <- function(measure) {
  client |>
  select(product_type, installed_capacity, is_business, date) |>
  mutate(date = ymd(date),
         year = year(date),
```

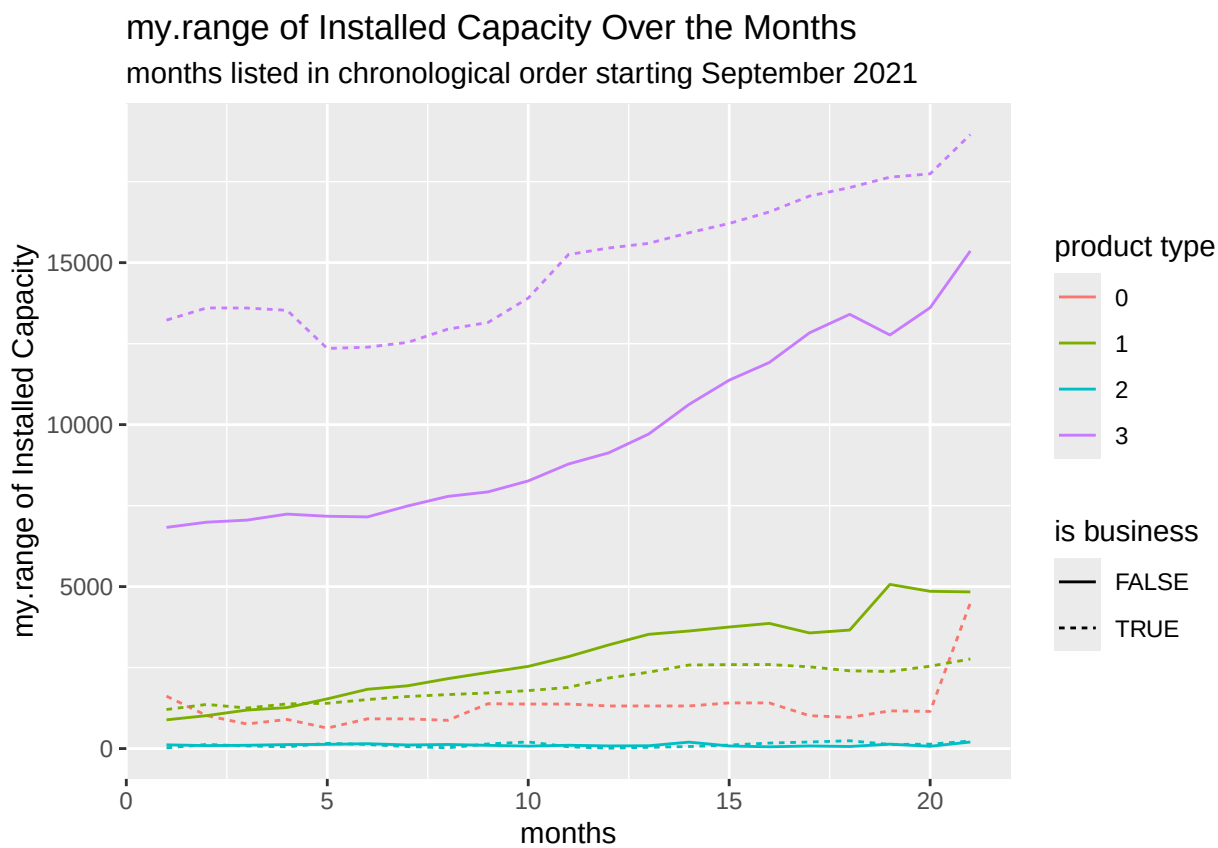
```r
         month = month(date)
         ) |>
  group_by(year, month, product_type, is_business) |>
  summarize({{measure}}:= measure(installed_capacity), .groups = "drop") |>
  left_join(rank, by = c("year" = "year",
                         "month" = "month")) }

# applying the function
overtime_mean <- overtime_df(mean)
overtime_median <- overtime_df(median)
overtime_my.range <- overtime_df(my.range)

# function to graph how mean/median/range changes overtime

overtime_graph <- function(df, measure) {
  title <- rlang::englue("{{measure}} of Installed Capacity Over the Months")
  y <- rlang::englue("{{measure}} of Installed Capacity")
  df |>
  ggplot(aes(x = chron_month, y = {{measure}}, color = as.factor(product_type), linetype = as.factor(is_
  geom_line() +
  labs(title = title,
       x = "months",
       y = y,
       subtitle = "months listed in chronological order starting September 2021",
       color = "product type",
       linetype = "is business"
       )
}

# applying function above
overtime_graph(overtime_mean, mean)
```




```
overtime_graph(overtime_median, median)
```




```
overtime_graph(overtime_my.range, my.range)
```



It appears that the mean installed capacity for business clients per product type is still generally greater than the mean installed capacity for non-business clients per product type when observed over the months.

The same holds true for the median considering product type 3. However, for product types 1 and 2, the leading curve fluctuates between the business clients and the non-business clients, which is not surprising, since for the previous bar graph, the values for those product types versus whether or not the client was a business were close in value.

Lastly, for the range of installed capacity, business clients thoroughly dominated for product type 3. This means that for product 3, the maximum and minimum installed capacity values differed more greatly for business clients. For product type 1, the non-business clients dominated after about month 5, and for product 2, the range of installed capacities for non-business and business clients stayed generally the same.

Analysis 3: Gas vs Electricity Prices

Are gas prices and electricity prices related? Does the date (for example month, day, year or a combination of month/day/year) matter?

Module(s) highlighted: Module 2, 3, 4

I first created summary data frames where I found the average gas and electricity prices per day.

```
# find average gas price and electricity price
summarized_gas_prices<- gas_prices |>
  mutate(avg_gas_price = (highest_price_per_mwh + lowest_price_per_mwh)/2) |>
  select(forecast_date, avg_gas_price, data_block_id)
```

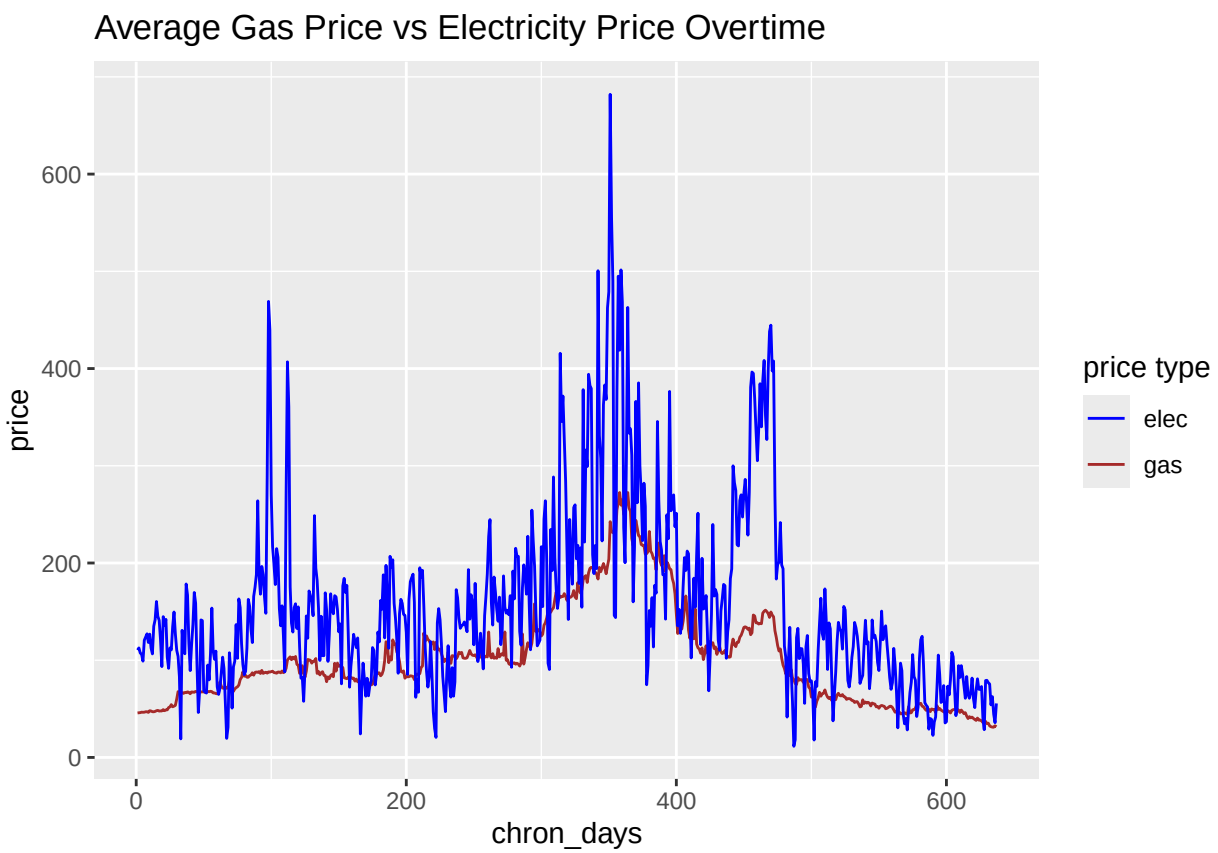
```r
summarized_elec_prices<- electricity_prices |>
  group_by(data_block_id) |>
  summarize(avg_electricity_price = mean(euros_per_mwh) )
```

I then used left_join to combine the gas and electricity summary data frames. Within the frame, I created columns specifying the year, month, and day of the week.

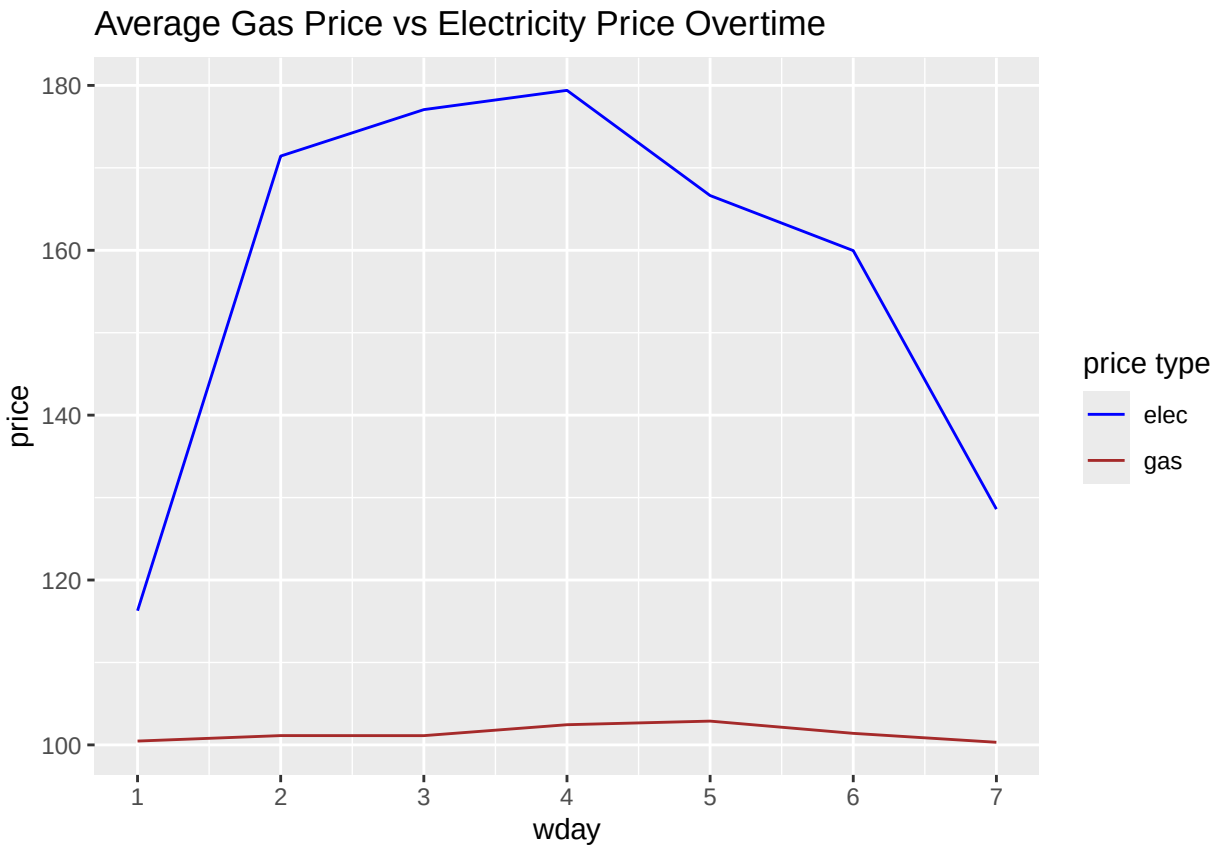
```r
# combine gas and electricity data frames
# create new columns for year, month, week

gas_elec_df<- summarized_gas_prices |>
  left_join(summarized_elec_prices, by = "data_block_id") |>
  mutate(chron_days = data_block_id) |>
  mutate(forecast_date = ymd(forecast_date),
         year = year(forecast_date),
         month = month(forecast_date),
         wday = wday(forecast_date)
         )
```

Using the data frame created above, I created a function to graph how the prices changed depending on the day of the week or month, and also how it changed chronologically by the day.

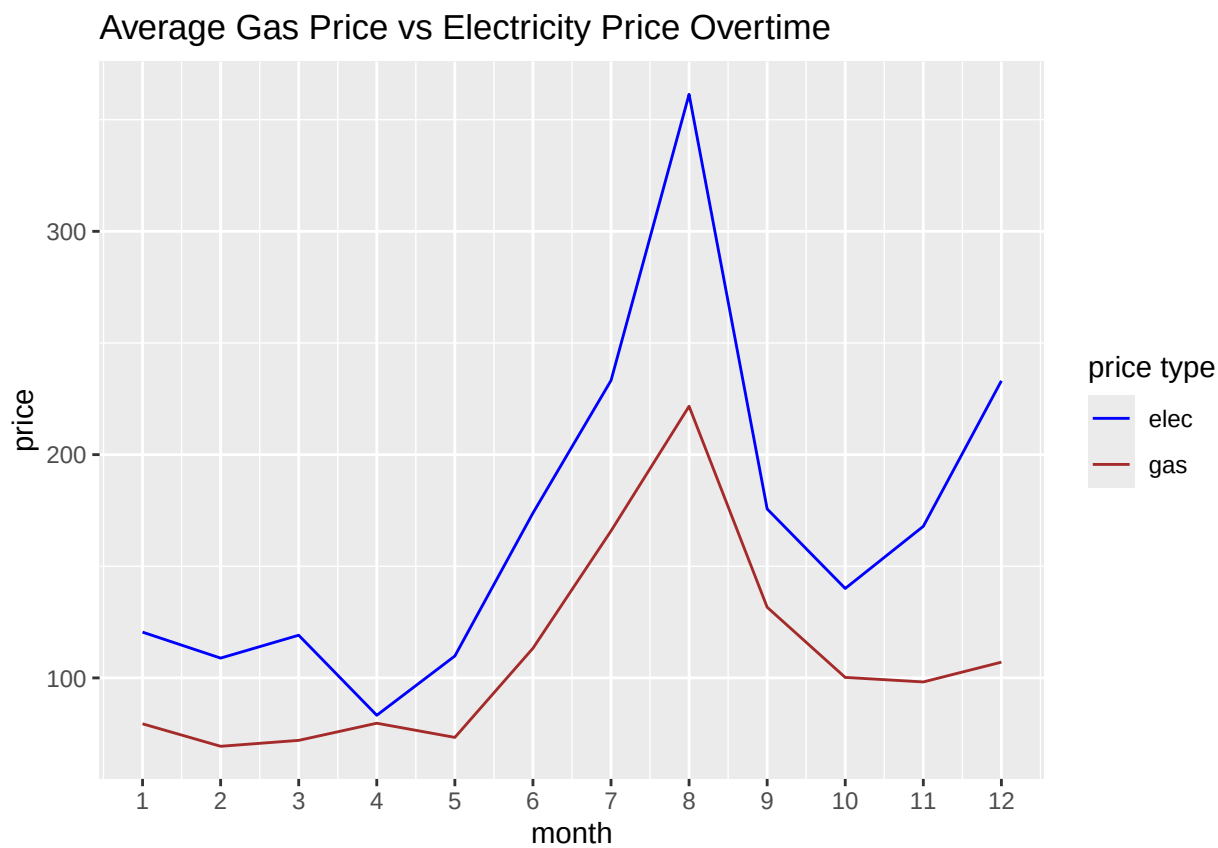
```r
graph_price_overtime <- function(time, n){

   x <- rlang::englue("{{time}}")
  colors <- c("gas" = "brown", "elec" = "blue")

  gas_elec_df |>
  group_by({{time}}) |>
  summarize(avg_gas_price = mean(avg_gas_price),
            avg_electricity_price = mean(avg_electricity_price)) |>
  ggplot() +
  geom_line(aes(x = {{time}}, y = avg_gas_price, color = "gas"))+
  geom_line(aes(x = {{time}}, y = avg_electricity_price, color = "elec")) +
  scale_x_continuous(n.breaks = n)+
  labs(title = "Average Gas Price vs Electricity Price Overtime",
       x = x,
       y = "price",
       color = "price type"
       )+
  scale_color_manual(values = colors)

}

# apply function above
graph_price_overtime(chron_days, 5)
```




```
graph_price_overtime(wday, 7)
```




```
graph_price_overtime(month, 12)
```

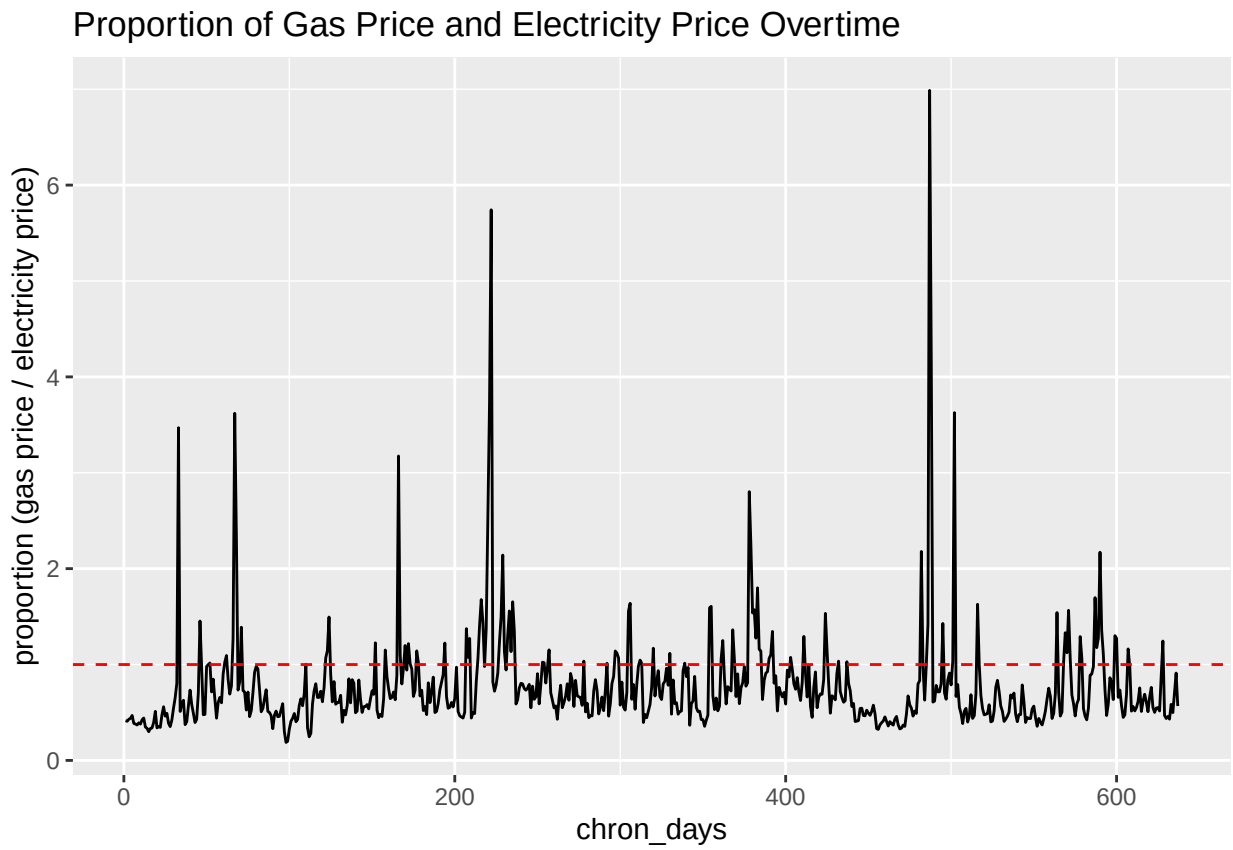


For the graph reporting the means of each type of price by the chronological day, each curve seems to follow roughly the same trend in the sense that their notable peaks, albeit observed at different magnitudes, occur at roughly the same time stamps. The same observation can be made for the graph reporting the means by month. Both price types exhibit their highest peaks on the month of August.

I decided to expand my analysis by investigating how the proportion between the gas price and electricity price changed depending on the day of the week, the month, and how it changed chronologically by the day.

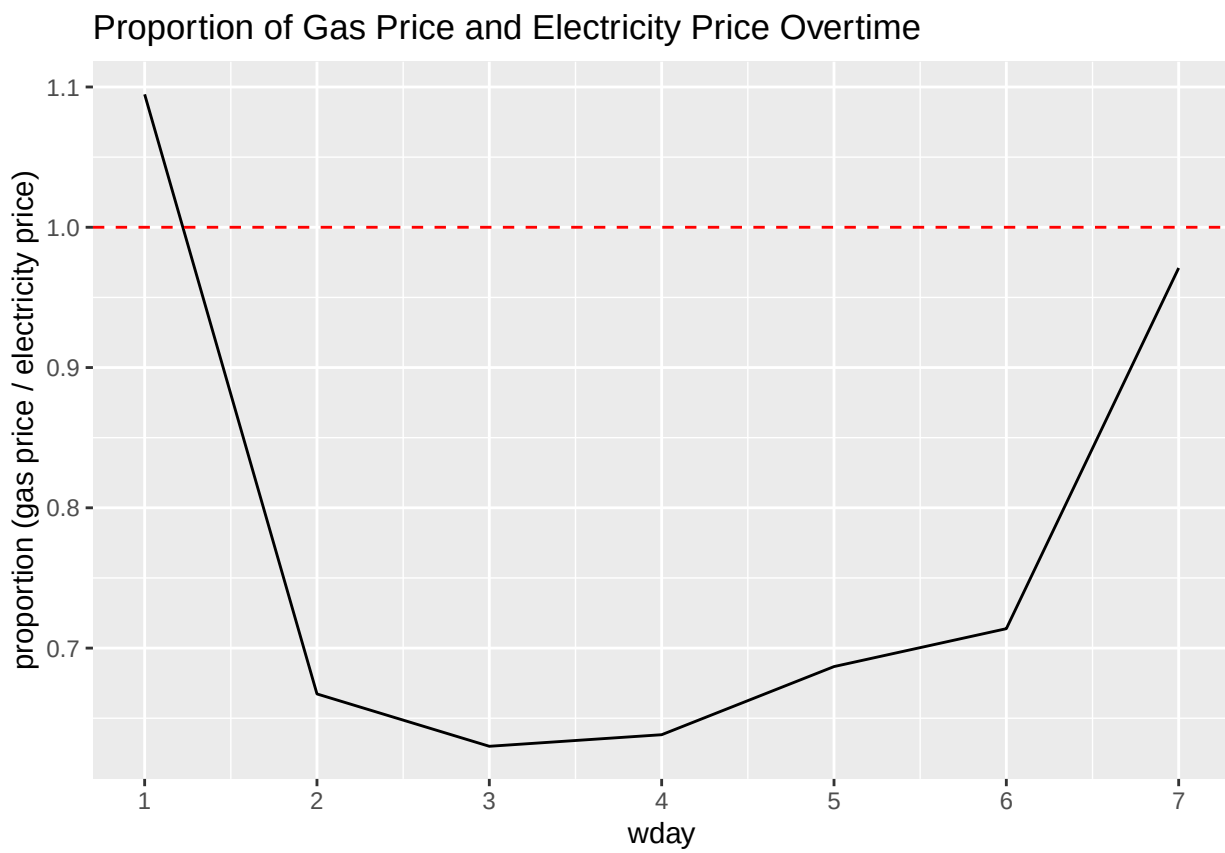
```
# function to graph how proportion changes
graph_price_prop_overtime <- function(time, n) {
  x <- rlang::englue("{{time}}")
  gas_elec_df |>
  mutate(prop_gas_elec = avg_gas_price / avg_electricity_price)|>
    group_by({{time}}) |>
    summarize(prop_gas_elec = mean(prop_gas_elec)) |>
    ggplot() +
    geom_line(aes(x = {{time}}, y = prop_gas_elec), color = "black") +
    scale_x_continuous(n.breaks = n) +
    geom_hline(yintercept=1, linetype="dashed", color = "red")+

  labs(title = "Proportion of Gas Price and Electricity Price Overtime",
       x = x,
       y = "proportion (gas price / electricity price)"
       )

}
```

```r
# applying the function above
graph_price_prop_overtime(chron_days, 5)
```




```r
graph_price_prop_overtime(month, 12)
```

```
graph_price_prop_overtime(wday, 7)
```



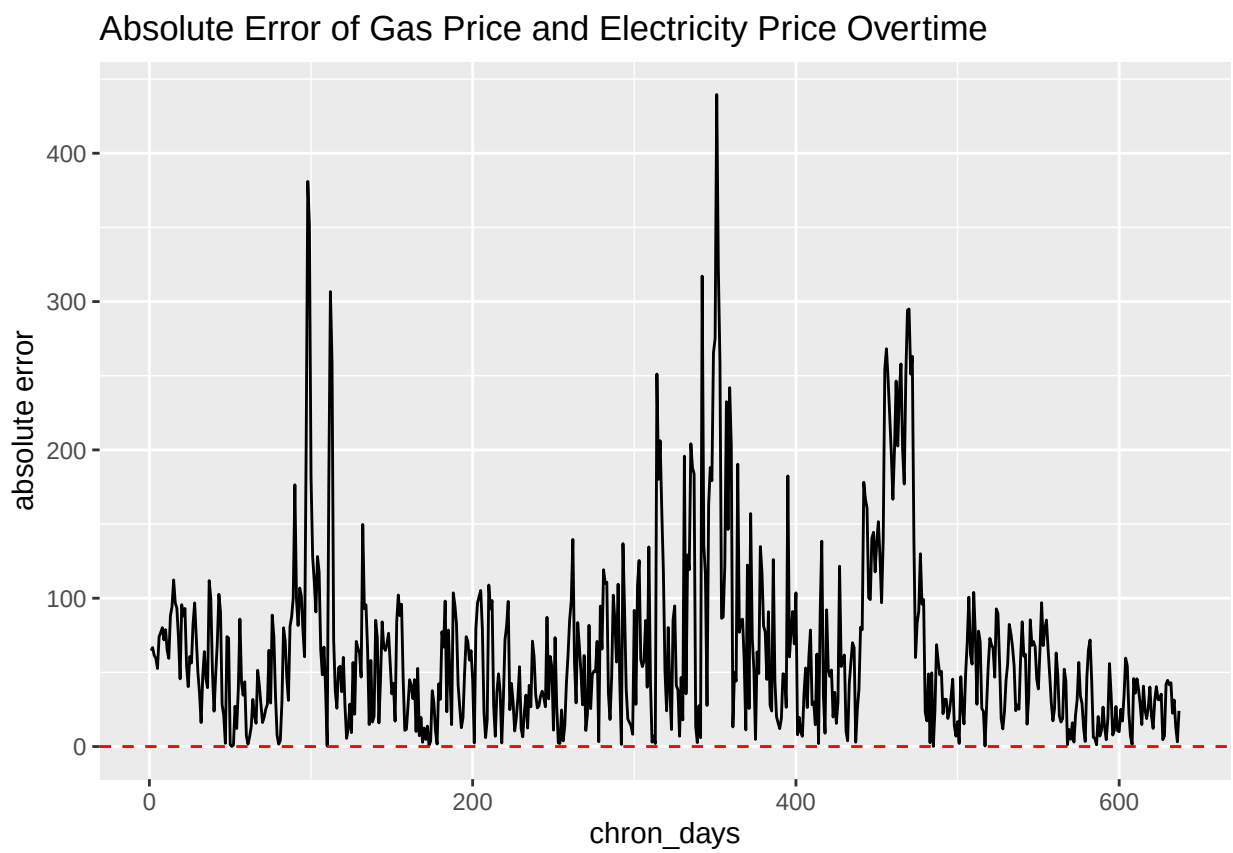
For these graphs, I added a line to mark where the proportion equaled to one, marking where the prices would be equal. When observing the graph of chronological days, I noticed a plentiful number of peaks touching the marker line. I also noticed how the prices seemed to be the most equal in value on the weekends, as demonstrated on the graph reporting the days of the week. The tendency for the prices to be equal on the weekends could contribute to the explanation behind why the curve for the chronological days graph exhibits numerous peaks near the marker line.

Compare the accuracy of the forecasted gas prices versus the forecasted electricity prices.

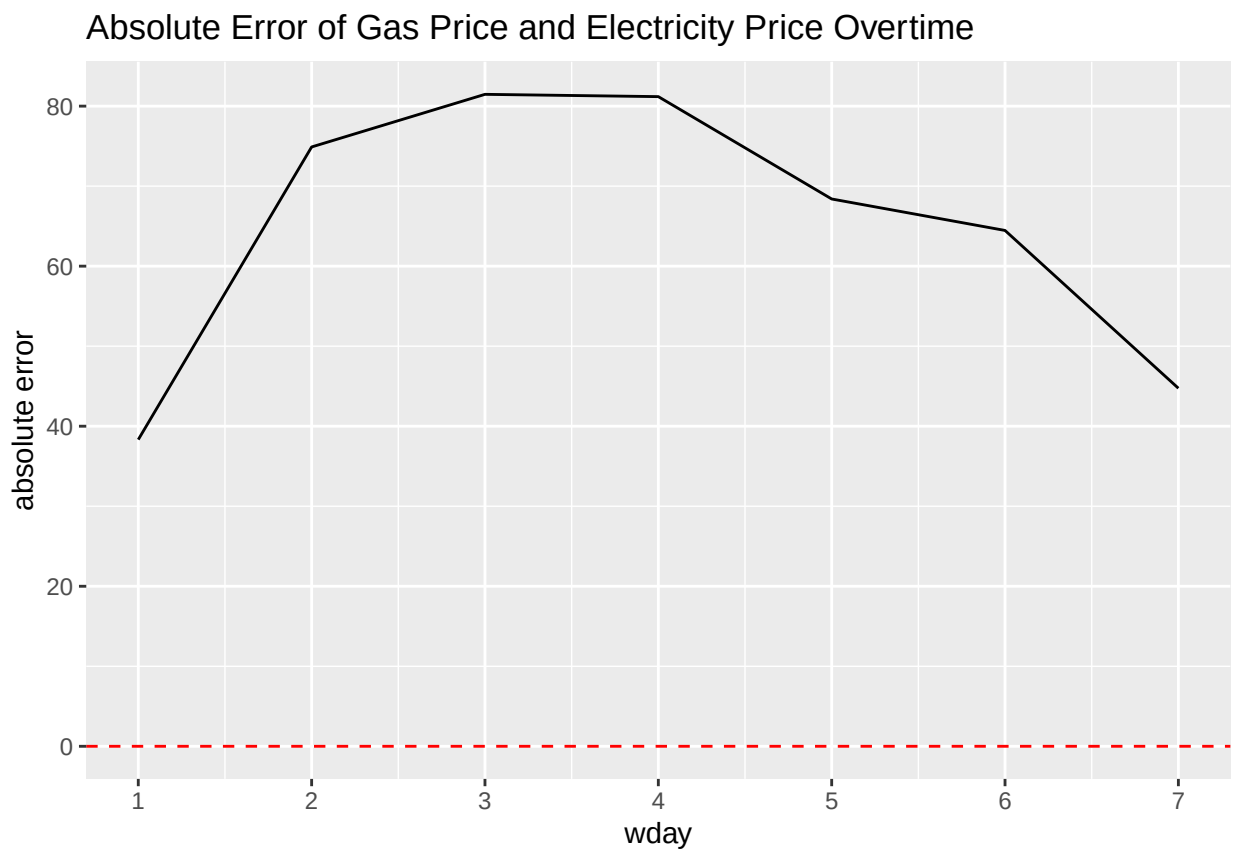
To answer the next part of the question, I created a function to compute the difference between the gas prices and electricity prices per day, then graph the relation overtime chronologically by the day, and also depending on the weekday and the month.

```
# compare the absolute error between the gas prices and electricity prices

abs_error_overtime <- function(time, n){
x <- rlang::englue("{{time}}")

gas_elec_df |>
  mutate(abs_error = abs(avg_electricity_price - avg_gas_price)) |>
  group_by({{time}}) |>
  summarize(abs_error = mean(abs_error)) |>
  ggplot() +
  geom_line(aes(x = {{time}}, y = abs_error), color = "black") +
  scale_x_continuous(n.breaks = n) +
  geom_hline(yintercept=0, linetype="dashed", color = "red")+
```

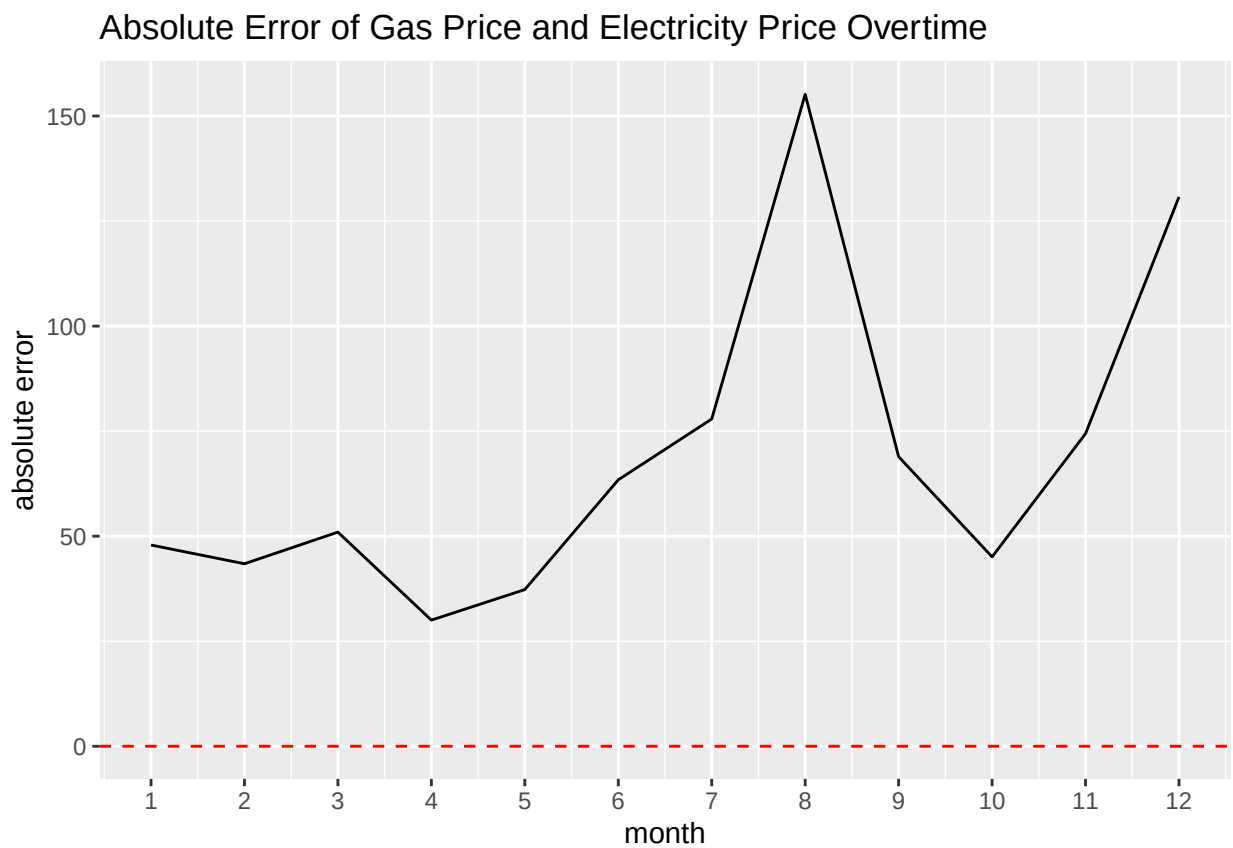
```r
  labs(title = "Absolute Error of Gas Price and Electricity Price Overtime",
       x = x,
       y = "absolute error"
       )

}

# graphing
abs_error_overtime(chron_days, 5)
```




```r
abs_error_overtime(wday, 7)
```




```
abs_error_overtime(month, 12)
```



The results elicited similar insights from my analysis with their proportions. For the weekdays graph, the absolute error is less on the weekends, so it seems that the prices tend to be closer in value on the weekends. This could contribute to the explanation why the chronological days graph seeing many dips toward y = 0, where the absolute error is 0. As for the months, it appears that the prices are more similar on April and October, while they deviate the most on August and December.

Analysis 4: Consumption Amount

Module(s) highlighted: Module 1, 4

How does the consumption amount of each county vary per time of day? Are high consumption amounts correlated with a certain time of day?

I started by filtering out the rows so they only included observations associated with consumption values. I then created columns to report the time of day. I then created a function to find the overall mean, median, and sum of the consumption to use for later computations.

```
# function to find the overall measure of consumption
measure.df <- function(measure) {

  train |>
    filter(is_consumption == 1) |>
    mutate(overall_measure_consumption = measure(target, na.rm = TRUE)) |>
    select(overall_measure_consumption) |>
```

```r
    slice_head()
}

mean.df <- measure.df(mean)
median.df <- measure.df(median)
sum.df <- measure.df(sum)

# using function to find overall measure of consumption
overall_mean_consumption <- mean.df$overall_measure_consumption[1]
overall_median_consumption <-median.df$overall_measure_consumption[1]
overall_sum_consumption <- sum.df$overall_measure_consumption[1]

overall_mean_consumption
```

```
## [1] 460.7081
```

```r
overall_median_consumption
```

```
## [1] 108.8965
```

```r
overall_sum_consumption
```

```
## [1] 464813907
```

I created a data frame reporting the mean, median, and percentage of consumption per group specified by county and time of day. To answer the second part of the problem, I created a column specifying whether or not each row exhibited a high consumption value.

```r
# data frame reporting mean/median/percentage consumption and high consumption boolean for counties per

county_consumption <- train |>
  filter(is_consumption == 1) |>
  mutate(datetime = ymd_hms(datetime),
         hour = hour(datetime),
         time_of_day = case_when(
           hour < 12 ~ "morning",
           hour >= 12 & hour < 18 ~ "afternoon",
           hour >= 18 ~ "evening")
        ) |>
  group_by(county, time_of_day) |>
  summarize(mean_consumption = mean(target, na.rm = TRUE),
            median_consumption = median(target, na.rm = TRUE),
            sum_consumption = sum(target, na.rm = TRUE))|>
  mutate(high_consumption_mean = case_when(
              mean_consumption >= overall_mean_consumption ~ "TRUE",
              mean_consumption < overall_mean_consumption ~ "FALSE"),
    high_consumption_median = case_when(
              median_consumption >= overall_median_consumption ~ "TRUE",
              median_consumption < overall_median_consumption ~ "FALSE"),
    percent_consumption = sum_consumption / overall_sum_consumption,
        ) |>
  ungroup()
```

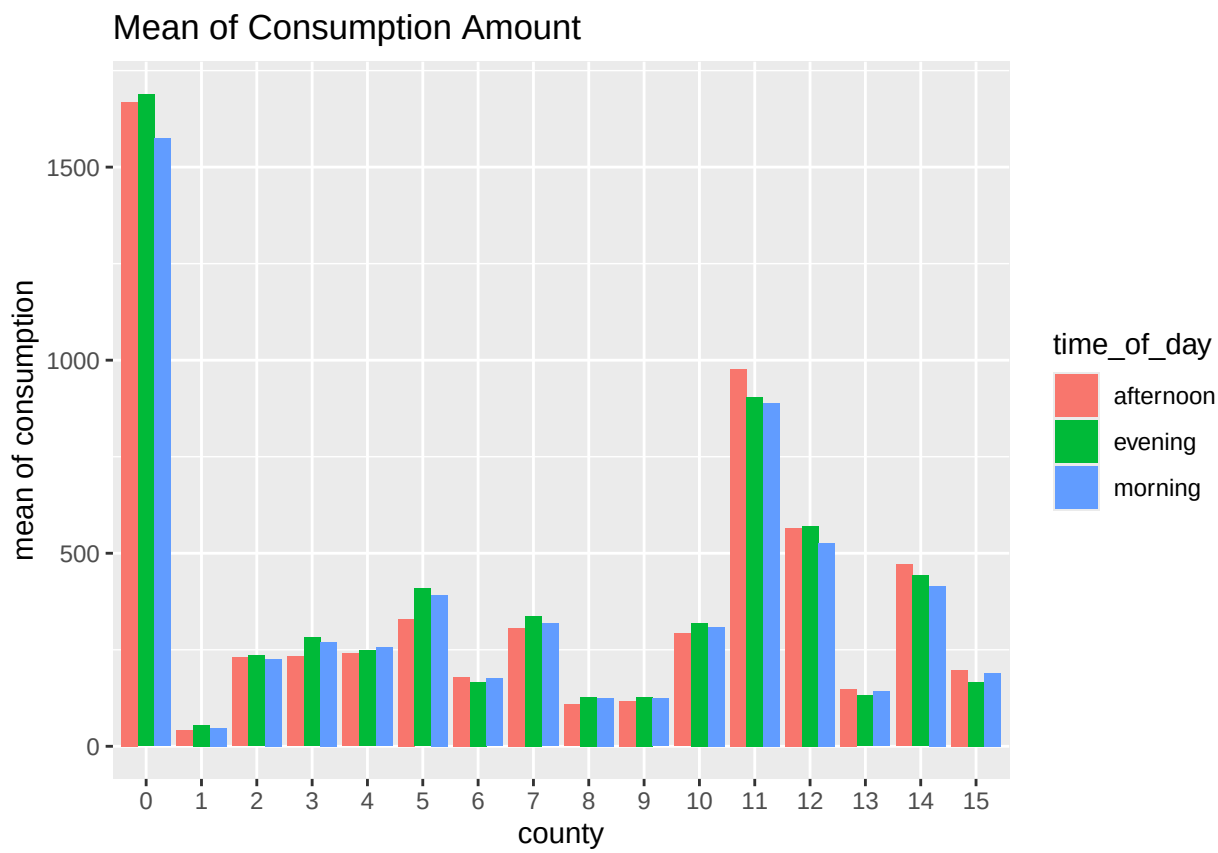
```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by county and time_of_day.
## i Output is grouped by county.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(county, time_of_day))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
county_consumption
```

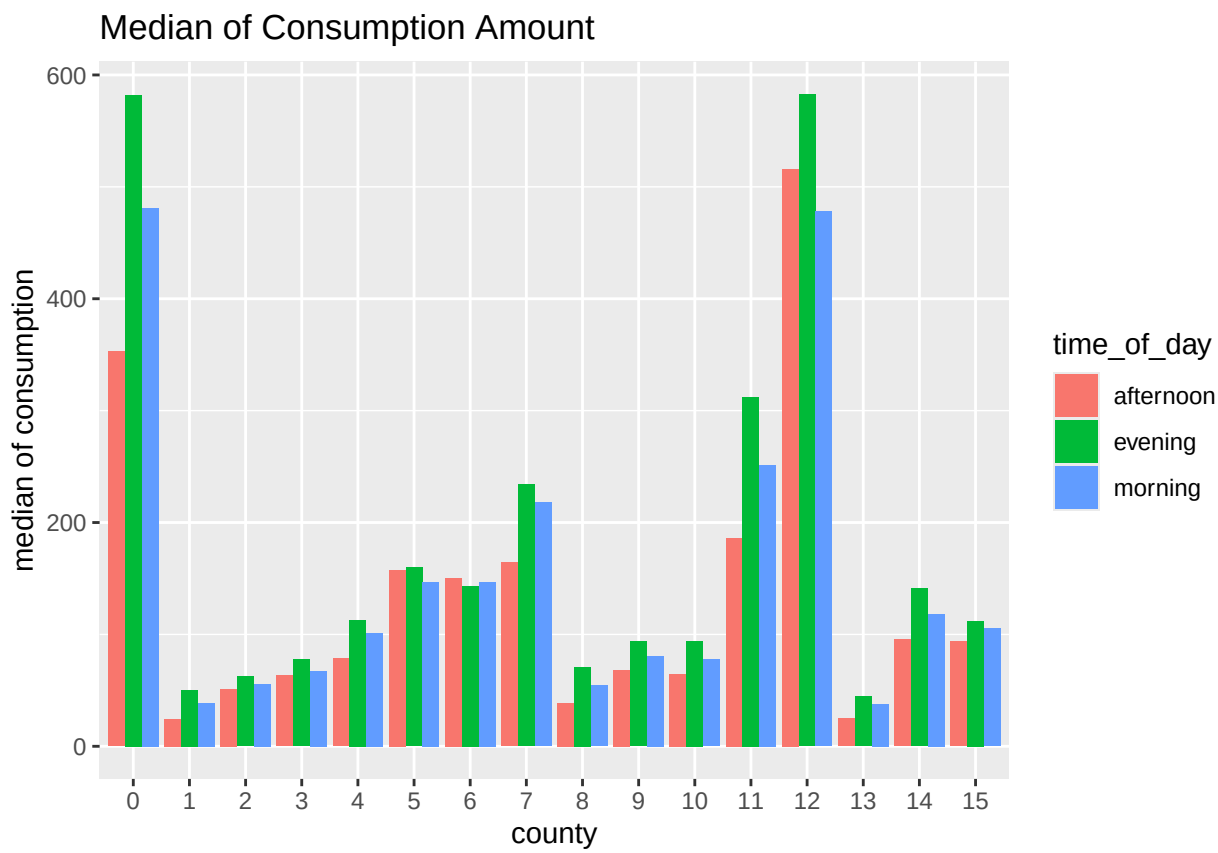
```
## # A tibble: 48 x 8
##    county time_of_day mean_consumption median_consumption sum_consumption
##     <int> <chr>                  <dbl>              <dbl>           <dbl>
##  1      0 afternoon             1668.               353.        44408671.
##  2      0 evening               1689.               582.        44941813.
##  3      0 morning               1574.               481.        83763323.
##  4      1 afternoon               40.9               24.3         470055.
##  5      1 evening                 54.5               50.5         625825.
##  6      1 morning                 47.3               38.9        1085625.
##  7      2 afternoon              230.                51.2        3318755.
##  8      2 evening                235.                62.5        3384599.
##  9      2 morning                226.                55.7        6491145.
## 10      3 afternoon              232.                63.1        3555608.
## # i 38 more rows
## # i 3 more variables: high_consumption_mean <chr>,
## #   high_consumption_median <chr>, percent_consumption <dbl>
```

I then graphed the mean, median, and percentage of consumption amount per county and time of day.

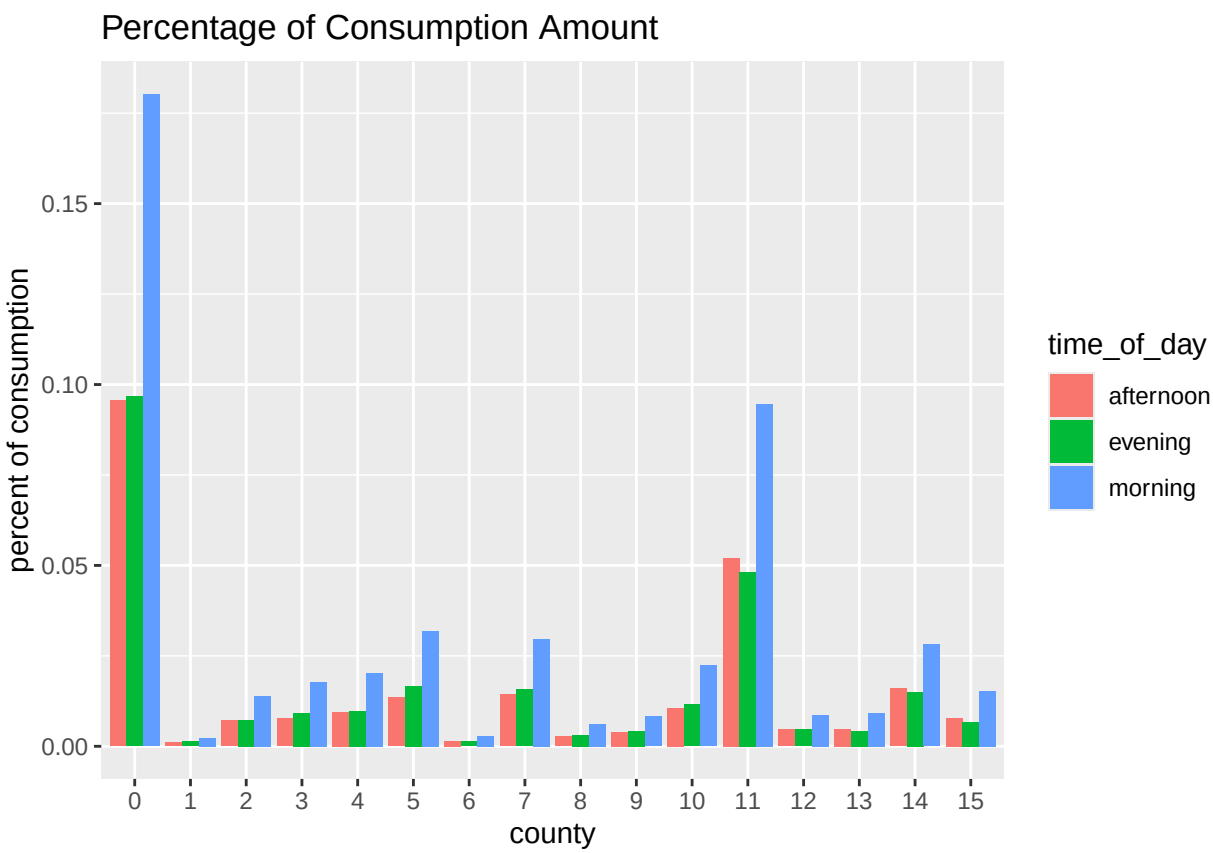
```
# graph of mean consumption
county_consumption |>
  ggplot(aes(x = as.factor(county),
             y = mean_consumption,
             fill = time_of_day)) +
  geom_col(position = "dodge")+

  labs(title = "Mean of Consumption Amount",
       x = "county",
       y = "mean of consumption")
```




```r
# graph of median consumption
county_consumption |>
  ggplot(aes(x = as.factor(county),
             y = median_consumption,
             fill = time_of_day)) +
  geom_col(position = "dodge")+

  labs(title = "Median of Consumption Amount",
       x = "county",
       y = "median of consumption")
```




```r
# graph of percentage of consumption
county_consumption |>
  ggplot(aes(x = as.factor(county),
             y = percent_consumption,
             fill = time_of_day)) +
  geom_col(position = "dodge") +

  labs(title = "Percentage of Consumption Amount",
       x = "county",
       y = "percent of consumption")
```



The mean consumption amount for the evening exhibits a trend of being slightly larger compared to the other times of days for each county. When observing the median graph, this trend is even more pronounced. However, this trend is not observed in the percentage of consumption graph, where for each county, the consumption percent for the morning is significantly higher compared to the rest of the days.

To answer the next question, I created a data frame that used the county_consumption data frame to count the number of county/time of day groups that exhibited average consumption values higher than the overall average. Whether or not a consumption value was high was determined by whether or not it was greater than the mean or median of the overall consumption amount.

```
#counting high consumption occurences with respect to mean
mean_consumption_boolean <- county_consumption |>
  group_by(time_of_day, high_consumption_mean) |>
  summarize(n = n())
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by time_of_day and high_consumption_mean.
## i Output is grouped by time_of_day.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(time_of_day, high_consumption_mean))` for
##   per-operation grouping (`?dplyr::dplyr_by`) instead.
```

```
#counting high consumption occurrences with respect to median
median_consumption_boolean<- county_consumption |>
  group_by(time_of_day, high_consumption_median) |>
  summarize(n = n())
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by time_of_day and high_consumption_median.
## i Output is grouped by time_of_day.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(time_of_day, high_consumption_median))` for
##   per-operation grouping (`?dplyr::dplyr_by`) instead.
```

```r
#combining both counts
consumption_boolean <- mean_consumption_boolean |>
  left_join(median_consumption_boolean,
            by = c("time_of_day" = "time_of_day",
                   "high_consumption_mean" = "high_consumption_median"))|>
  rename(n.mean = 'n.x',
         n.median = 'n.y',
         high_consumption = 'high_consumption_mean')

mean_consumption_boolean
```

```
## # A tibble: 6 x 3
## # Groups:   time_of_day [3]
##   time_of_day high_consumption_mean     n
##   <chr>       <chr>                 <int>
## 1 afternoon   FALSE                    12
## 2 afternoon   TRUE                      4
## 3 evening     FALSE                    13
## 4 evening     TRUE                      3
## 5 morning     FALSE                    13
## 6 morning     TRUE                      3
```

```r
median_consumption_boolean
```

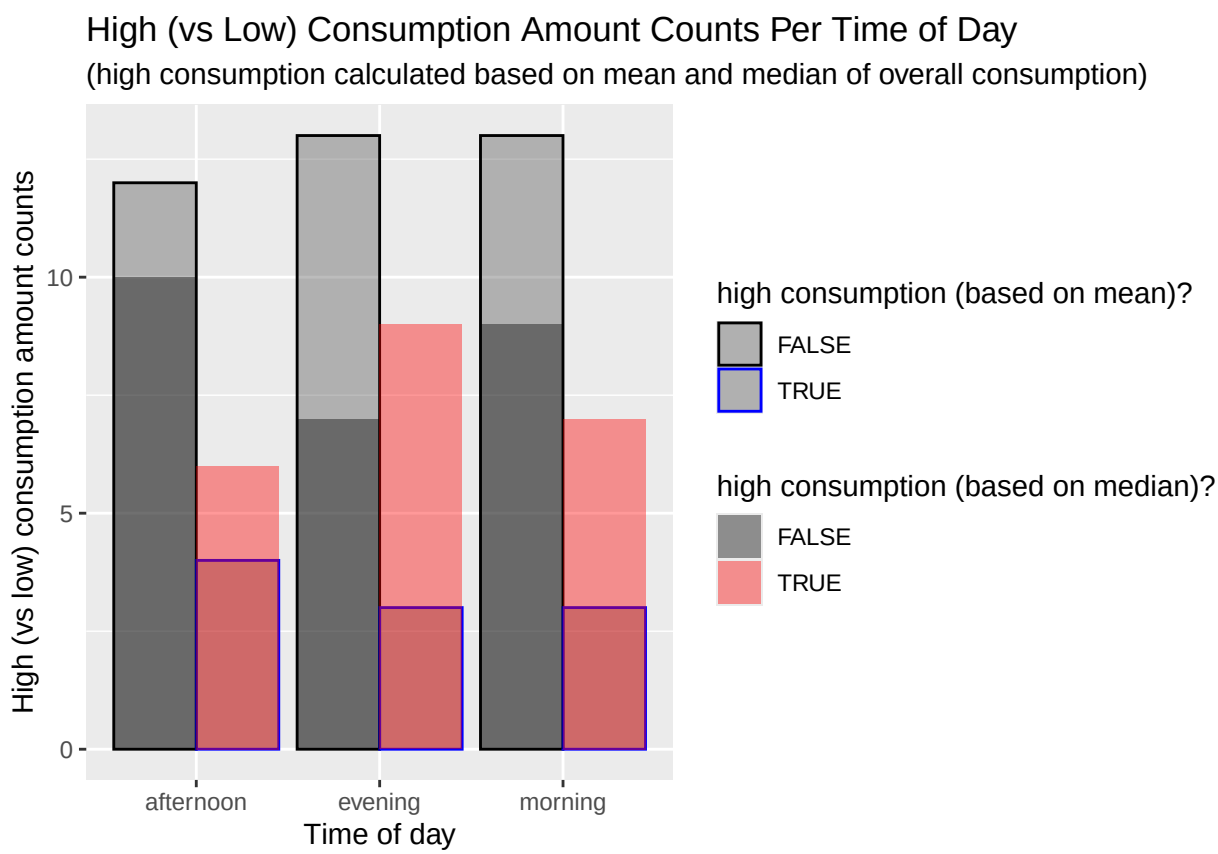
```
## # A tibble: 6 x 3
## # Groups:   time_of_day [3]
##   time_of_day high_consumption_median     n
##   <chr>       <chr>                   <int>
## 1 afternoon   FALSE                      10
## 2 afternoon   TRUE                        6
## 3 evening     FALSE                       7
## 4 evening     TRUE                        9
## 5 morning     FALSE                       9
## 6 morning     TRUE                        7
```

```r
consumption_boolean
```

```
## # A tibble: 6 x 4
## # Groups:   time_of_day [3]
##   time_of_day high_consumption n.mean n.median
##   <chr>       <chr>             <int>    <int>
## 1 afternoon   FALSE                12       10
## 2 afternoon   TRUE                  4        6
## 3 evening     FALSE                13        7
## 4 evening     TRUE                  3        9
## 5 morning     FALSE                13        9
## 6 morning     TRUE                  3        7
```

I then plotted the number of high vs low consumption values per time of day.

```
# data derived after calculating the average consumption amount for each county per time of day,
# then determining whether or not the values were high
consumption_boolean |>
  ggplot() +
  geom_col(aes(x = time_of_day, y = n.mean, color = high_consumption), position = "dodge", alpha = 0.4)
  geom_col(aes(x = time_of_day, y = n.median, fill = high_consumption), position = "dodge", alpha = 0.4
  labs(title = "High (vs Low) Consumption Amount Counts Per Time of Day",
       x = "Time of day",
       y = "High (vs low) consumption amount counts",
       subtitle = "(high consumption calculated based on mean and median of overall consumption)",
       fill = "high consumption (based on median)?",
       color = "high consumption (based on mean)?"
         )+
  scale_fill_manual(values = c("FALSE" = "black", "TRUE" = "red")) +
  scale_color_manual(values = c("FALSE" = "black", "TRUE" = "blue"))
```



When observing the high consumption count based on the mean, there isn't much of a variation per time of day, but afternoon exhibits the highest count. As for the high consumption based on the median, the evening exhibits the highest count compared to the other times of days. This observation agrees with the observation made in the previous bar graph reporting the median consumption amounts per county per time of day. There might exist a correlation between high consumption amounts and the evening time, but the previous bar graph reporting percentage of consumption amounts per county per time of day weakens that suggestion. A stronger claim would be to state that the morning contributes the highest percentage of consumption.